Spring Boot Coffee Stock Inventory REST API

Project Features / Deliverables

We will build these REST endpoints:

Method	Endpoint	Purpose
GET	/api/coffees	Get all coffees
GET	/api/coffees/{id}	Get coffee by ID
POST	/api/coffees	Add new coffee
PUT	/api/coffees/{id}	Update coffee
DELETE	/api/coffees/{id}	Delete coffee
GET	/api/coffees/search?name=Latte	Search coffee
GET	/api/coffees/category/{category}	Get by category
PATCH	/api/coffees/{id}/stock	Update stock quantity

Architecture:

Client / Postman

|

v

Controller

|

v

Service Interface

|

v

Service Implementation

|

v

ArrayList<Coffee>

No JPA, no repository/database, and no application.properties database configuration.

---

1. Create Spring Boot Project

Use Spring Initializr:

Spring Initializr

Select:

Project : Maven

Language : Java

Spring Boot : Latest stable version

Packaging : Jar

Java : 17 or later

Dependency:

Spring Web

Project name:

coffee-stock-api

---

2. Project Structure

Create this structure:

```
coffee-stock-api
|
├── src/main/java/com/example/coffeestock
|   |
|   ├── CoffeeStockApplication.java
|   |
|   ├── controller
```

```
│   │   └── CoffeeController.java
│   │
│   ├── model
│   │   └── Coffee.java
│   │
│   ├── service
│   │   ├── CoffeeService.java
│   │   └── CoffeeServiceImpl.java
│   │
│   └── exception
│       └── CoffeeNotFoundException.java
│
└── src/main/resources
    └── application.properties
```

---

3. Main Spring Boot Class

CoffeeStockApplication.java

```
package com.example.coffeestock;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class CoffeeStockApplication {

    public static void main(String[] args) {

        SpringApplication.run(CoffeeStockApplication.class, args);
```

```
    }
}
```

---

4. Create Coffee Model

Create:

model/Coffee.java

```
package com.example.coffeestock.model;
public class Coffee {
    private int id;
    private String name;
    private String category;
    private double price;
    private int stock;

    public Coffee() {
    }

    public Coffee(int id, String name, String category,
                  double price, int stock) {

        this.id = id;
        this.name = name;
        this.category = category;
        this.price = price;
        this.stock = stock;
    }
```

```
public int getId() {
    return id;
}

public void setId(int id) {
    this.id = id;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public String getCategory() {
    return category;
}

public void setCategory(String category) {
    this.category = category;
}

public double getPrice() {
    return price;
}
```

```
    public void setPrice(double price) {
        this.price = price;
    }

    public int getStock() {
        return stock;
    }

    public void setStock(int stock) {
        this.stock = stock;
    }
}
```

The JSON representation will look like:

```
{
   "id": 1,
   "name": "Cappuccino",
   "category": "Hot Coffee",
   "price": 150.0,
   "stock": 25
}
```

---

5. Create Service Interface

Create:

service/CoffeeService.java

```
package com.example.coffeestock.service;

import com.example.coffeestock.model.Coffee;
```

```
import java.util.List;

public interface CoffeeService {

    List<Coffee> getAllCoffees();

    Coffee getCoffeeById(int id);

    Coffee addCoffee(Coffee coffee);

    Coffee updateCoffee(int id, Coffee coffee);

    void deleteCoffee(int id);

    List<Coffee> searchByName(String name);

    List<Coffee> getByCategory(String category);

    Coffee updateStock(int id, int stock);
}
```

This interface defines what our Coffee service can do.

---

6. Create Custom Exception

Create:

exception/CoffeeNotFoundException.java

```
package com.example.coffeestock.exception;

public class CoffeeNotFoundException extends RuntimeException {
```

```
    public CoffeeNotFoundException(String message) {
        super(message);
    }
}
```

For example:

Coffee with ID 10 not found

will be returned when an invalid ID is requested.

---

7. Create Service Implementation

This is the most important class because the **ArrayList acts as our temporary data store**.

Create:

service/CoffeeServiceImpl.java

```
package com.example.coffeestock.service;

import com.example.coffeestock.exception.CoffeeNotFoundException;
import com.example.coffeestock.model.Coffee;

import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.List;

@Service
public class CoffeeServiceImpl implements CoffeeService {

    private final List<Coffee> coffeeList = new ArrayList<>();
```

```
public CoffeeServiceImpl() {

    coffeeList.add(
        new Coffee(1, "Cappuccino", "Hot Coffee", 150, 25)
    );

    coffeeList.add(
        new Coffee(2, "Latte", "Hot Coffee", 180, 20)
    );

    coffeeList.add(
        new Coffee(3, "Espresso", "Hot Coffee", 120, 30)
    );

    coffeeList.add(
        new Coffee(4, "Cold Coffee", "Cold Coffee", 160, 15)
    );
}

@Override
public List<Coffee> getAllCoffees() {

    return coffeeList;
}

@Override
public Coffee getCoffeeById(int id) {
```

```
    for (Coffee coffee : coffeeList) {

        if (coffee.getId() == id) {
            return coffee;
        }
    }

    throw new CoffeeNotFoundException(
        "Coffee with ID " + id + " not found"
    );
}

@Override
public Coffee addCoffee(Coffee coffee) {

    coffeeList.add(coffee);

    return coffee;
}

@Override
public Coffee updateCoffee(int id, Coffee coffee) {

    Coffee existingCoffee = getCoffeeById(id);

    existingCoffee.setName(coffee.getName());
    existingCoffee.setCategory(coffee.getCategory());
```

```
    existingCoffee.setPrice(coffee.getPrice());
    existingCoffee.setStock(coffee.getStock());

    return existingCoffee;
}

@Override
public void deleteCoffee(int id) {

    Coffee coffee = getCoffeeById(id);

    coffeeList.remove(coffee);
}

@Override
public List<Coffee> searchByName(String name) {

    List<Coffee> result = new ArrayList<>();

    for (Coffee coffee : coffeeList) {

        if (coffee.getName()
                .toLowerCase()
                .contains(name.toLowerCase())) {

            result.add(coffee);
        }
    }
```

```
    return result;
}

@Override
public List<Coffee> getByCategory(String category) {

    List<Coffee> result = new ArrayList<>();

    for (Coffee coffee : coffeeList) {

        if (coffee.getCategory()
                .equalsIgnoreCase(category)) {

            result.add(coffee);
        }
    }

    return result;
}

@Override
public Coffee updateStock(int id, int stock) {

    Coffee coffee = getCoffeeById(id);

    coffee.setStock(stock);
```

```
        return coffee;
    }
}
```

Important

There is no:

@Repository

and no:

JpaRepository

because we aren't using a database.

Instead:

private final List<Coffee> coffeeList = new ArrayList<>();

is our temporary inventory.

---

8. Create REST Controller

Create:

controller/CoffeeController.java

```
package com.example.coffeestock.controller;

import com.example.coffeestock.model.Coffee;
import com.example.coffeestock.service.CoffeeService;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;
@RestController
@RequestMapping("/api/coffees")
```

```
public class CoffeeController {

    private final CoffeeService coffeeService;

    public CoffeeController(CoffeeService coffeeService) {
        this.coffeeService = coffeeService;
    }

    // GET ALL
    @GetMapping
    public ResponseEntity<List<Coffee>> getAllCoffees() {

        return ResponseEntity.ok(
            coffeeService.getAllCoffees()
        );
    }
    // GET BY ID
    @GetMapping("/{id}")
    public ResponseEntity<Coffee> getCoffeeById(
            @PathVariable int id) {

        return ResponseEntity.ok(
            coffeeService.getCoffeeById(id)
        );
    }

    // ADD COFFEE
    @PostMapping
```

```
public ResponseEntity<Coffee> addCoffee(
        @RequestBody Coffee coffee) {

    Coffee savedCoffee =
            coffeeService.addCoffee(coffee);

    return new ResponseEntity<>(
            savedCoffee,
            HttpStatus.CREATED
    );
}

// UPDATE COFFEE
@PutMapping("/{id}")
public ResponseEntity<Coffee> updateCoffee(
        @PathVariable int id,
        @RequestBody Coffee coffee) {

    return ResponseEntity.ok(
        coffeeService.updateCoffee(id, coffee)
    );
}

// DELETE COFFEE
@DeleteMapping("/{id}")
public ResponseEntity<String> deleteCoffee(
        @PathVariable int id) {
```

```
    coffeeService.deleteCoffee(id);

    return ResponseEntity.ok(
        "Coffee deleted successfully"
    );
}

// SEARCH
@GetMapping("/search")
public ResponseEntity<List<Coffee>> searchCoffee(
        @RequestParam String name) {

    return ResponseEntity.ok(
        coffeeService.searchByName(name)
    );
}

// CATEGORY
@GetMapping("/category/{category}")
public ResponseEntity<List<Coffee>> getByCategory(
        @PathVariable String category) {

    return ResponseEntity.ok(
        coffeeService.getByCategory(category)
    );
}

// UPDATE STOCK
```

```
    @PatchMapping("/{id}/stock")
    public ResponseEntity<Coffee> updateStock(
            @PathVariable int id,
            @RequestParam int stock) {

        return ResponseEntity.ok(
            coffeeService.updateStock(id, stock)
        );
    }
}
```

---

9. application.properties

Since there is **no database**, keep it very simple.

spring.application.name=coffee-stock-api

server.port=8080

---

10. Complete API Flow

For example, when we execute:

GET /api/coffees/2

the request flows:

```
Postman
   |
   v
CoffeeController
   |
   | getCoffeeById(2)
   v
CoffeeService
```

|

v

CoffeeServiceImpl

|

v

ArrayList<Coffee>

|

v

Coffee object

|

v

JSON Response

---

11. Test Using Postman

Start the application.

You should see:

Tomcat started on port 8080

GET All Coffees

GET http://localhost:8080/api/coffees

Response:

```
[
   {
      "id": 1,
      "name": "Cappuccino",
      "category": "Hot Coffee",
      "price": 150.0,
      "stock": 25
   },
```

```
  {
    "id": 2,
    "name": "Latte",
    "category": "Hot Coffee",
    "price": 180.0,
    "stock": 20
  },
  {
    "id": 3,
    "name": "Espresso",
    "category": "Hot Coffee",
    "price": 120.0,
    "stock": 30
  },
  {
    "id": 4,
    "name": "Cold Coffee",
    "category": "Cold Coffee",
    "price": 160.0,
    "stock": 15
  }
]
```


---

12. Get Coffee By ID

GET http://localhost:8080/api/coffees/2

Response:


```
{
  "id": 2,
```

```
    "name": "Latte",
    "category": "Hot Coffee",
    "price": 180.0,
    "stock": 20
}
```


---

13. Add New Coffee

Use:

POST http://localhost:8080/api/coffees

Body → raw → JSON:


```
{
    "id": 5,
    "name": "Mocha",
    "category": "Hot Coffee",
    "price": 200,
    "stock": 18
}
```


Response:


```
{
    "id": 5,
    "name": "Mocha",
    "category": "Hot Coffee",
    "price": 200.0,
    "stock": 18
}
```


The object is added to:

coffeeList

---

14. Update Coffee

PUT http://localhost:8080/api/coffees/5

Body:

```
{
   "name": "Dark Mocha",
   "category": "Premium Coffee",
   "price": 220,
   "stock": 25
}
```

The existing coffee is updated.

---

15. Delete Coffee

DELETE http://localhost:8080/api/coffees/5

Response:

Coffee deleted successfully

---

16. Search Coffee

GET http://localhost:8080/api/coffees/search?name=Latte

Response:

```
[
   {
      "id": 2,
      "name": "Latte",
      "category": "Hot Coffee",
      "price": 180.0,
      "stock": 20
   }
]
```

The search is case-insensitive.

For example:

Latte

latte

LATTE

will all work.

---

17. Search By Category

GET http://localhost:8080/api/coffees/category/Hot Coffee

You may URL-encode the space:

GET http://localhost:8080/api/coffees/category/Hot%20Coffee

Response:


```
[
   {
      "id": 1,
      "name": "Cappuccino",
      "category": "Hot Coffee",
      "price": 150.0,
      "stock": 25
   },
   {
      "id": 2,
      "name": "Latte",
      "category": "Hot Coffee",
      "price": 180.0,
      "stock": 20
   },
   {
```

```
    "id": 3,
    "name": "Espresso",
    "category": "Hot Coffee",
    "price": 120.0,
    "stock": 30
  }
]
```


---

18. Update Stock

This is particularly useful for an inventory application.

PATCH http://localhost:8080/api/coffees/2/stock?stock=50

Response:


```
{
  "id": 2,
  "name": "Latte",
  "category": "Hot Coffee",
  "price": 180.0,
  "stock": 50
}
```


---

19. Exception Example

Try:

GET http://localhost:8080/api/coffees/100

Since ID 100 doesn't exist:

```
throw new CoffeeNotFoundException(
  "Coffee with ID " + id + " not found"
);
```

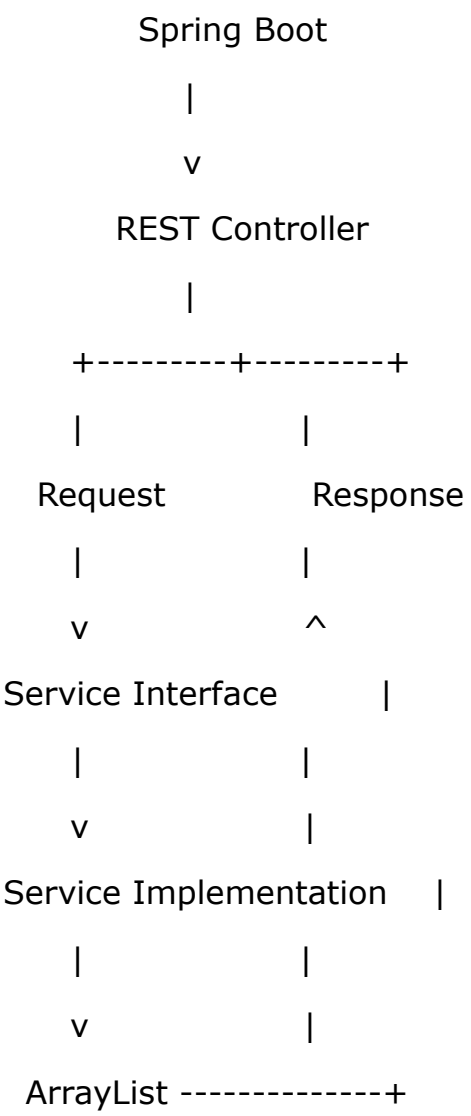
The application will return an error response.

For a beginner project, this demonstrates **custom exception handling**, without introducing global exception handling yet.

---

20. What Students Learn from This Project

This small project covers the complete basic REST API flow:



Spring concepts

- @SpringBootApplication
- @RestController
- @RequestMapping
- @GetMapping
- @PostMapping

- @PutMapping
- @PatchMapping
- @DeleteMapping
- @PathVariable
- @RequestParam
- @RequestBody
- Constructor Dependency Injection
- @Service
- Service interface
- Service implementation
- ResponseEntity
- HTTP status codes
- Custom exception

Java concepts

- Interface
- Implementation class
- ArrayList
- List
- Constructor
- Encapsulation
- Loops
- String operations

REST concepts

GET → Read

POST → Create

PUT → Update

PATCH → Partial Update

DELETE → Delete

One important limitation: because the inventory is stored in an ArrayList, **all data disappears when the Spring Boot application is restarted**. That's intentional for this version; the next natural evolution is to replace the ArrayList with a repository and database while keeping the Controller → Service architecture largely the same.